

All contents, audio, video, texts, and graphics are property of the Technische Universität Dresden and the chair for computer architecture if not marked otherwise. All rights are restricted. Without a written approval, you are not allowed to copy, change, distribute, or (re-)publish any of the work.

Sämtliche Inhalte, Audio, Video, Texte und Grafiken sind urheberrechtlich geschützt. Sie dürfen ohne vorherige schriftliche Genehmigung weder ganz noch auszugsweise kopiert verändert, modifiziert oder veröffentlicht werden.

© Technische Universität Dresden

Dr. Robert Schöne

APB 1029

☎ +49 351 463 42483

WIL A104

☎ +49 351 463 35450

<https://tu-dresden.de/zih/die-einrichtung/struktur/robert-schoene>

robert.schoene@tu-dresden.de

High Performance Computers and Their Programming (Preparation)

Hardware Basics

Here's what you know

An example

C

```
#include <stdio.h>

int main()
{
    int sum = 0;

    for ( int i = 0 ; i < 10 ; i++ )
        sum+=i;

    printf("%d\n",sum);

    return 0;
}
```

Python

```
sum=0

for i in range(10):
    sum+=i

print(sum)
```

How did you get this result?

- Read the instructions
- Calculate

Congratulations, you can calculate like a computer!

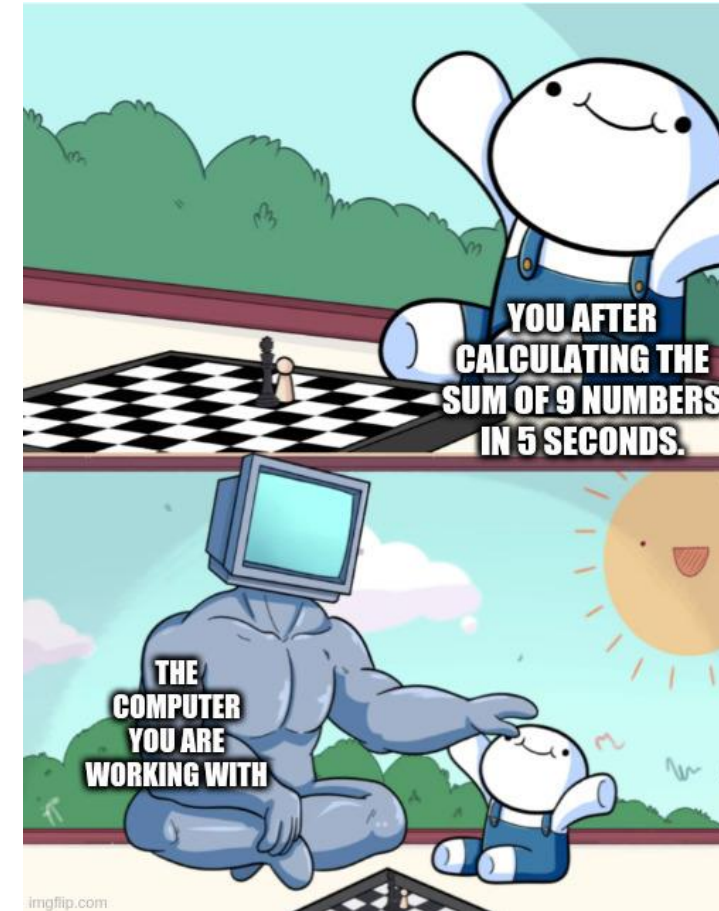


Your task:

What is printed at the command line?

You wish!

- That took way too long
- You should have done that in $< 0.01 \mu\text{s}$
- But you can listen to me, interpret the images that contain the code correctly, *and* execute the task afterwards
- So, you're fine 😊, just not optimal for numerical calculations



Let's have a look on what you did, and how a computer would do that

You

- Given:
 - List of ordered statements
- Interpretation:
 - Read statement
 - Understand statement
 - Execute statement
 - Read next statement
 - Understand next statement
 - Execute next statement
 - ...

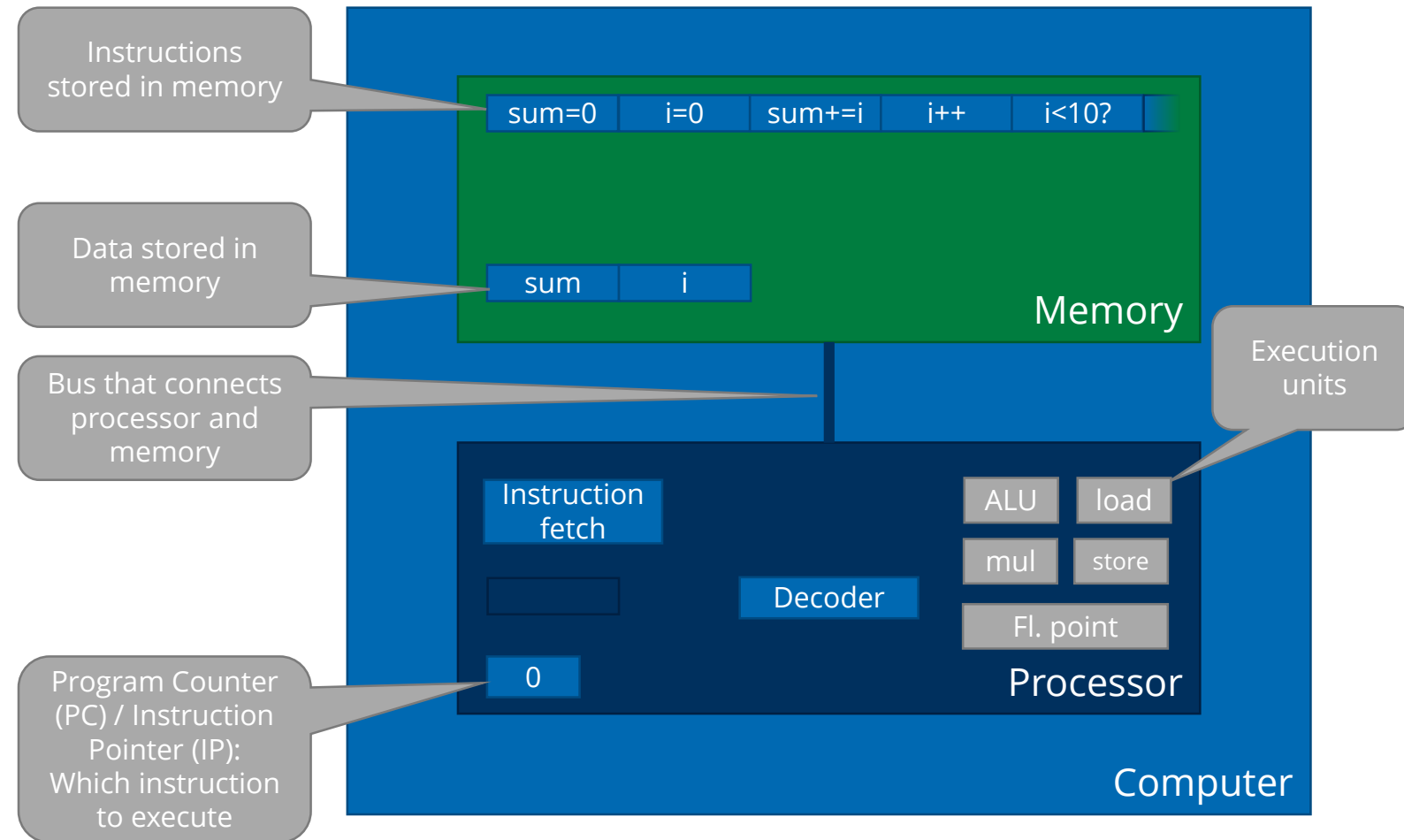
Computer

- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction
 - Decode instruction
 - Execute operation
 - Read data
 - Calculate
 - Write result
 - Read next instruction
 - ...

Things needed

Computer

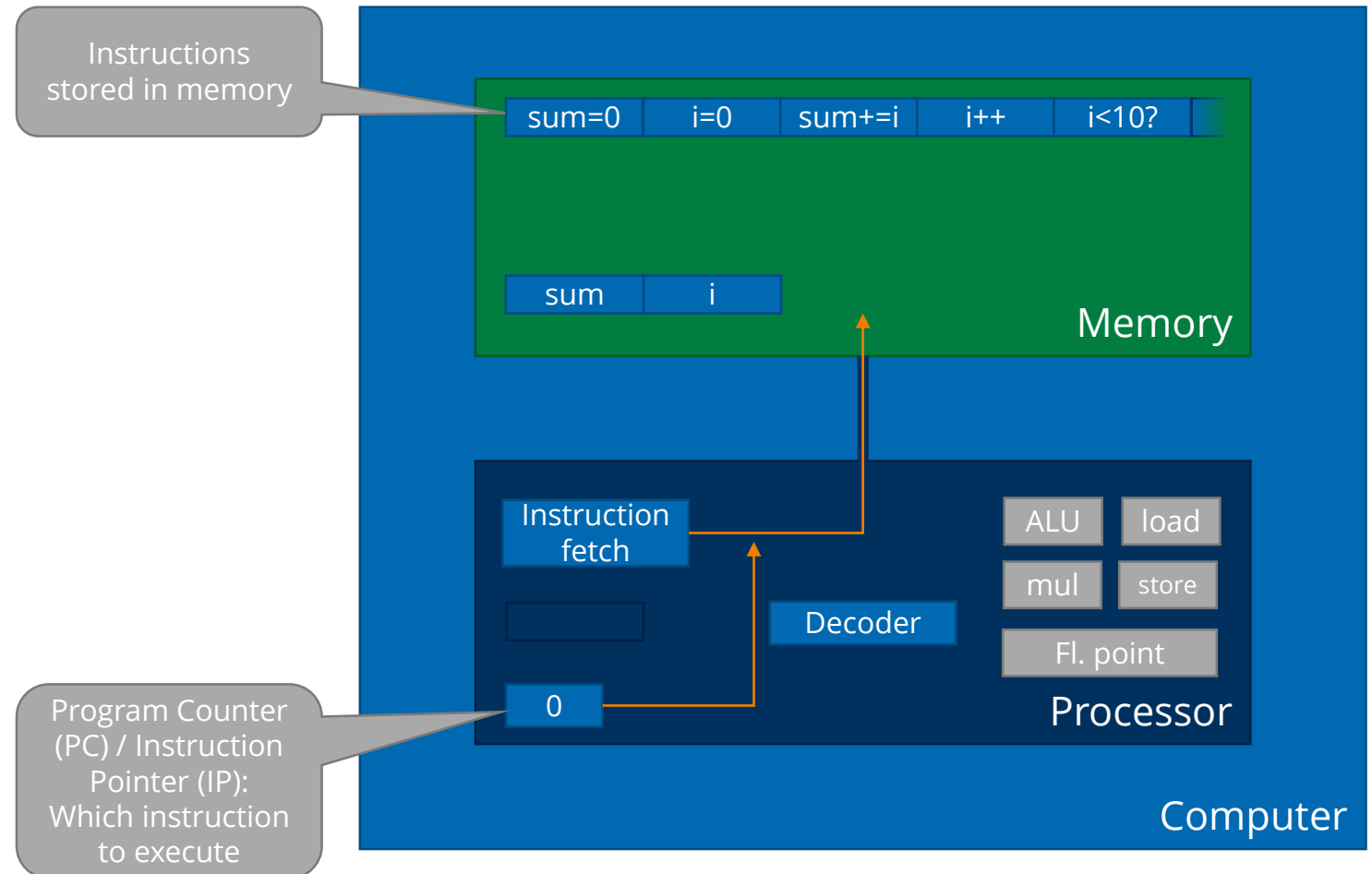
- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - Decode instruction 0
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Computer

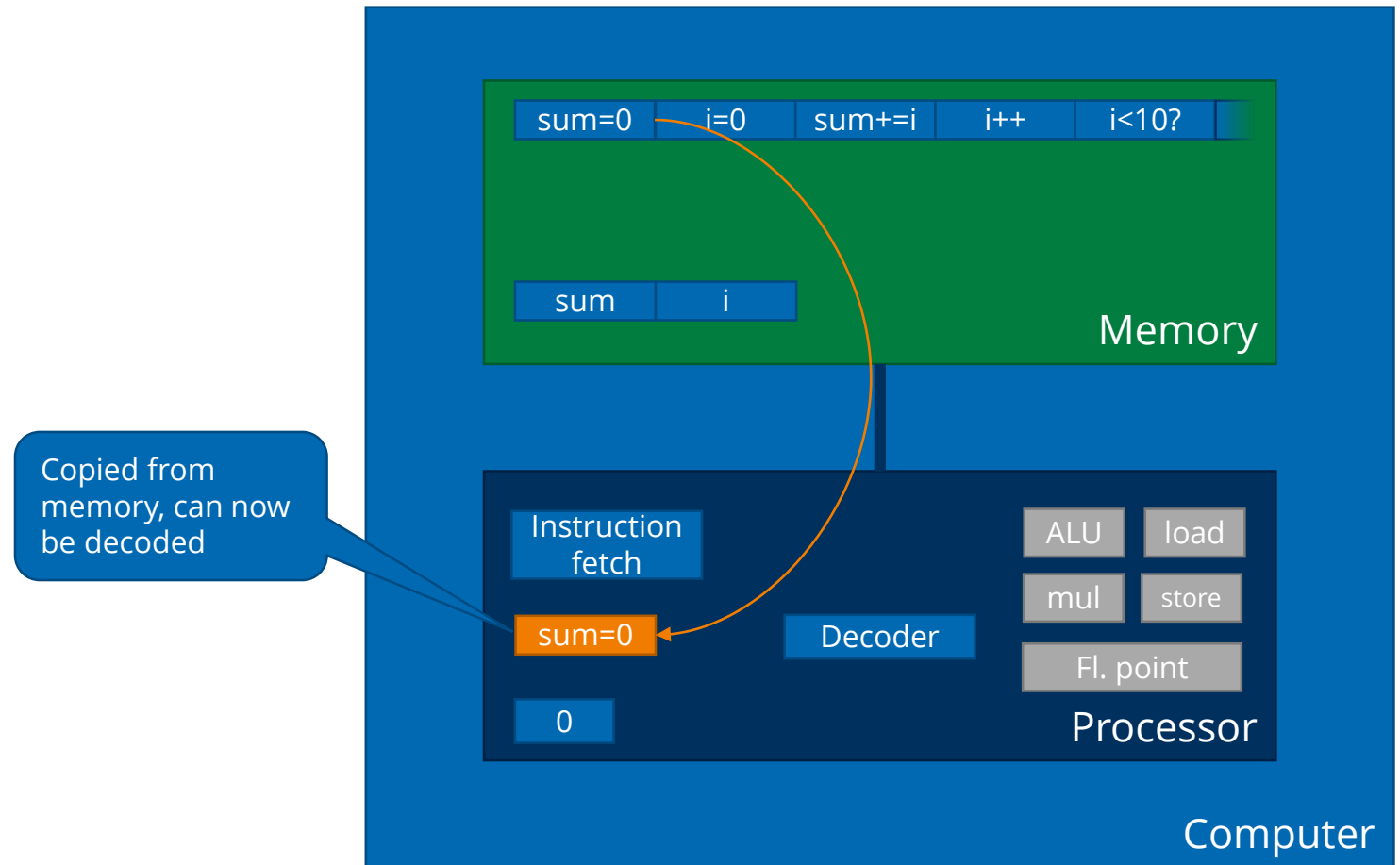
- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - Decode instruction 0
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Computer

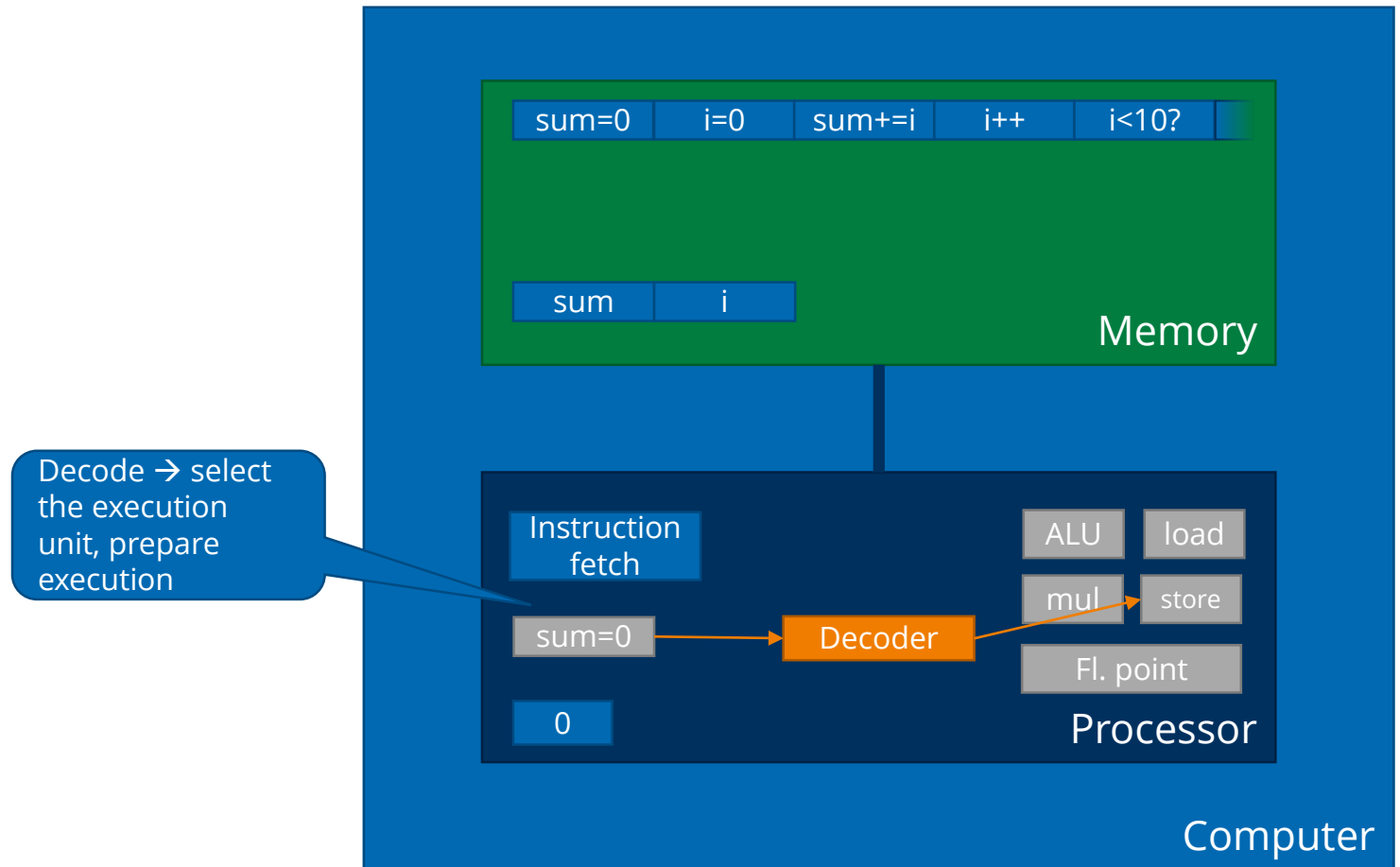
- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - Decode instruction 0
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Computer

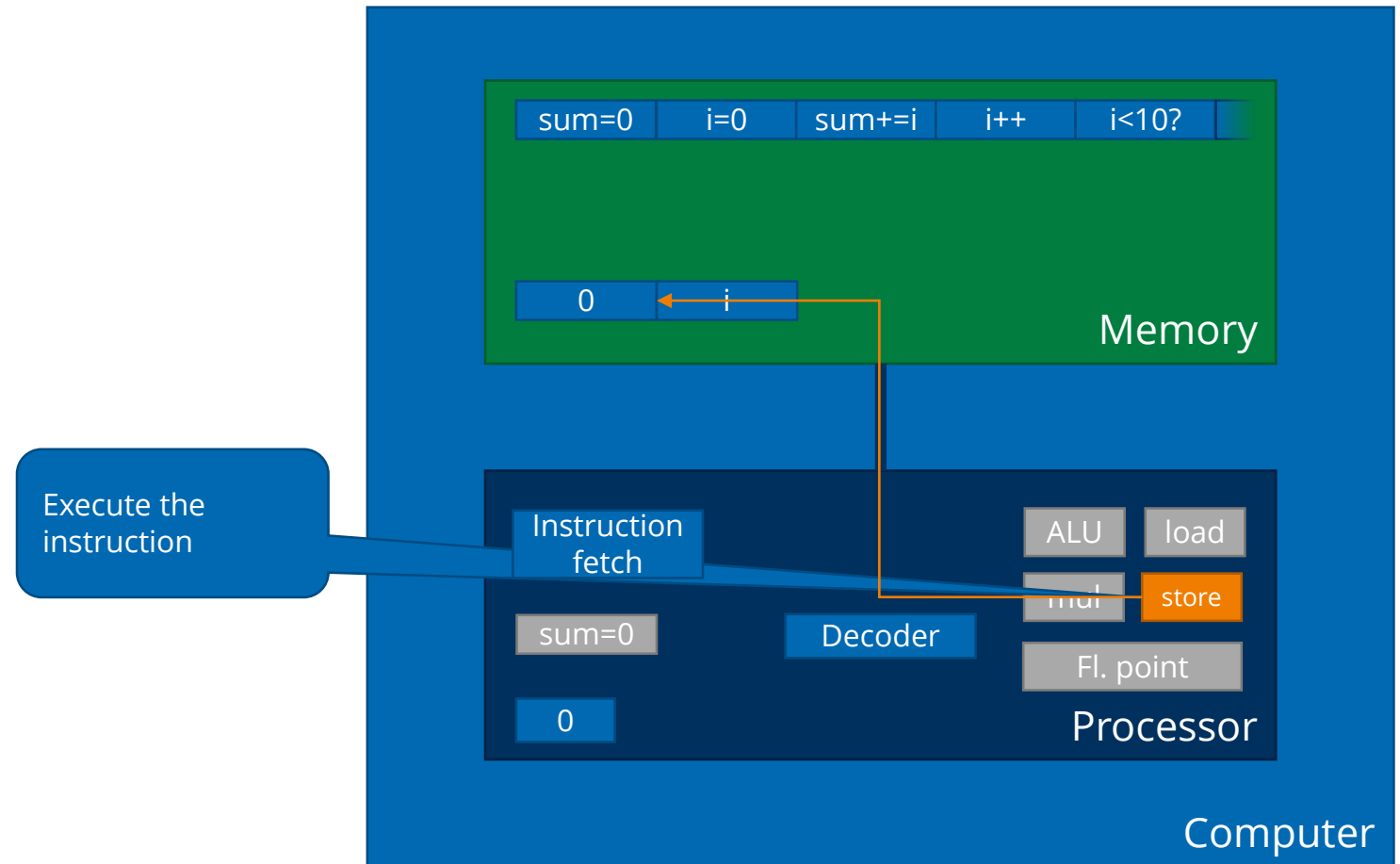
- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - **Decode instruction 0**
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Computer

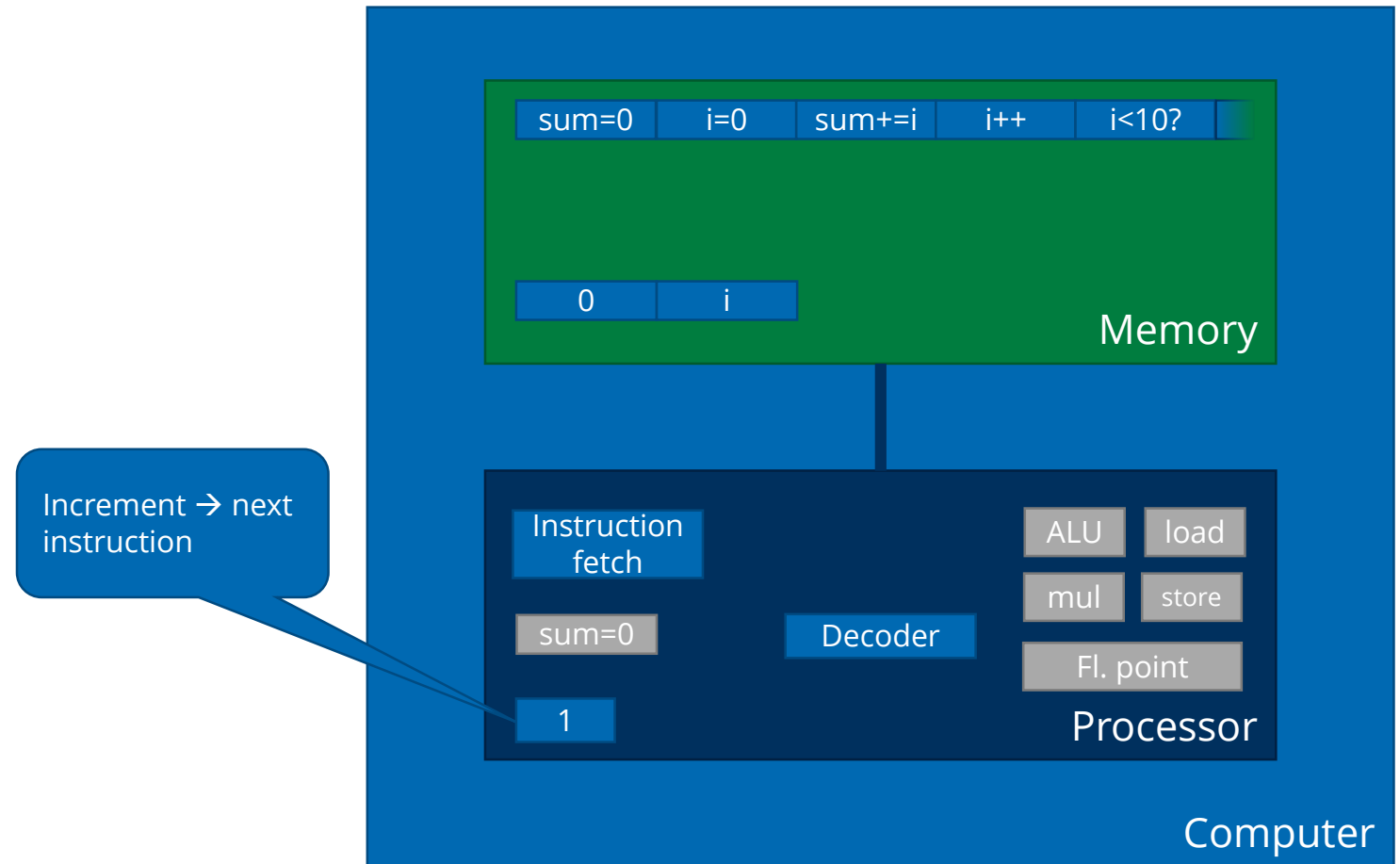
- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - Decode instruction 0
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Computer

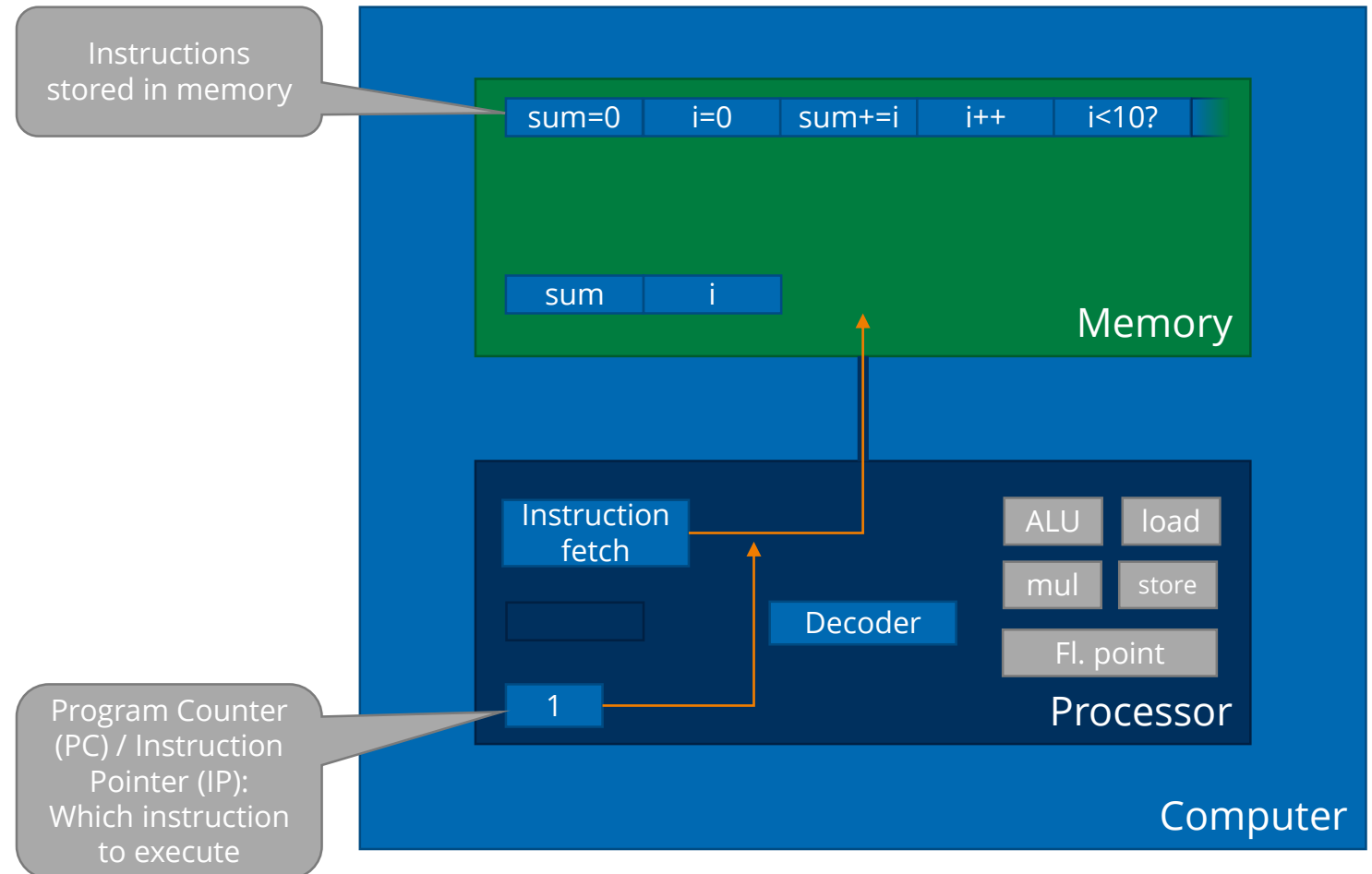
- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - Decode instruction 0
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Computer

- Given:
 - List of ordered instructions
- Interpretation:
 - Read instruction 0
 - Decode instruction 0
 - Execute operation 0
 - Read data
 - Calculate
 - Write result
 - Read instruction 1
 - ...



Things needed

Hardware

- Memory
 - Stores instructions
 - Stores data
- Bus
- Processor:
 - Program Counter
 - Execution Units
 - Registers (not depicted before)
- I/O bus (not depicted before)
 - For input/output devices like hard drives

Common Language / Instruction Set Architecture (ISA)

- Software uses this language
- Hardware understands this language
- Defines possible instructions
- Defines available registers
- Defines how to access memory
- Defines how to do I/O
- Examples: x86, ARM, RISC-V, RDNA

Things not depicted before: Registers

- Today, mostly General Purpose registers (integers, memory addresses)
- Usage depending on ISA, example:
 - `LOAD r1, (r2) # load from adress stored # in register 2 to reg. 1`
 - `ADD r3, r3, r1 # add to register 3`
 - `ADD r2, $8 # increase r2 by 8`
 - `CMP r2, r4 # compare with r4`
 - `B.LT $-20 # if < go back 20 bytes # (5 instructions)`
- Special registers for non-integers
- Program Counter can also be a register

Terms, you might hear

- Load/store architecture:
 - An ISA, where data cannot be used directly from memory, but has to be expliitely transferred from/to registers

— 3-Operand-Format

- Instructions with three operands, mostly:

$$r_{target} = r_{source1}(op)r_{source2}$$

— 2-Operand-Format

- Instructions with two operands, mostly:

$$r_{target/source1} = r_{target/source1}(op)r_{source2}$$

Things not depicted before: I/O devices and bus(es)

- Includes all sorts of long term storage
 - Hard drive
 - Flash drive
 - ...
- Includes all devices for user interaction
 - Keyboard
 - Mouse
 - Digitizer
 - ...
- Includes all devices to connect to the outside world
 - (W)LAN card, bluetooth chip, ...
- Includes all busses within the computer
 - SATA (Serial ATA)
 - PCIe (PCI express)
 - USB (Universal Serial Bus)

Hardware Optimizations

Basic Assumption

- Five steps of processing an instruction
 - Instruction Fetch
 - Fetch instruction from memory to processor
 - Instruction Decode
 - Decode instruction and select execution unit for the underlying operation, e.g. ALU
 - Execute
 - Execute the operation on the execution unit
 - Memory (optional)
 - Access memory
 - Why after „Execution“? Execution can calculate a memory address which would be used here
 - Writeback (optional)
 - Write result back to registers

How long would the example from before take?

- The loop can be realized with two instructions on some platforms
- Add + conditional jump (one of these can increase i implicitly)
- Sums up to 20 instructions
- Each of these has 5 steps
- → 100 cycles (= 50 ns on a 2 GHz processor)
- But we said „*You should have done that in $< 0.01 \mu\text{s}$* “, which is less than 50 ns
- How do we get there?

```
#include <stdio.h>

int main()
{
    int sum = 0;

    for ( int i = 0 ; i < 10 ; i++ )
        sum+=i;

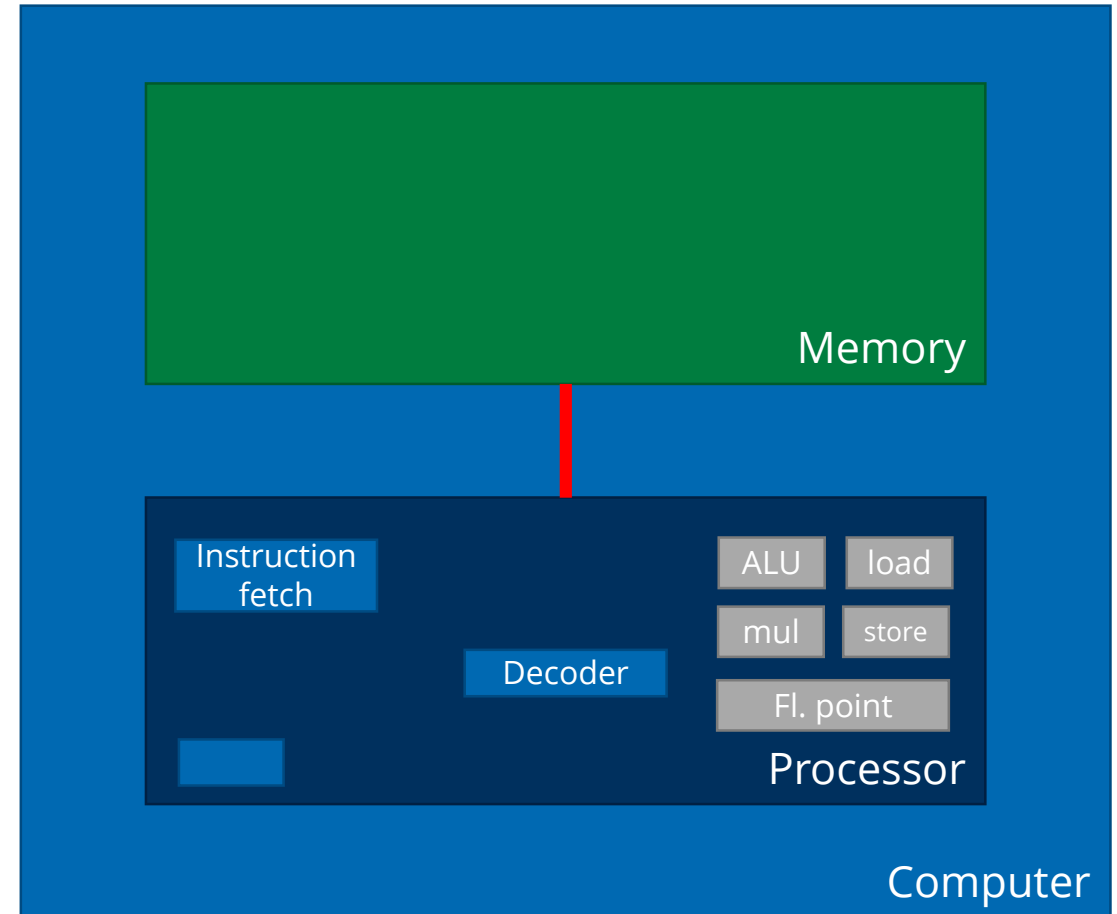
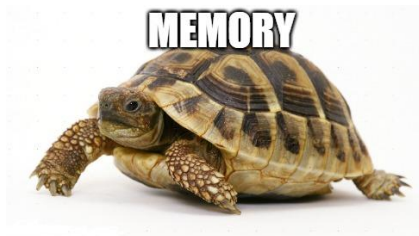
    printf("%d\n",sum);

    return 0;
}
```

Caches

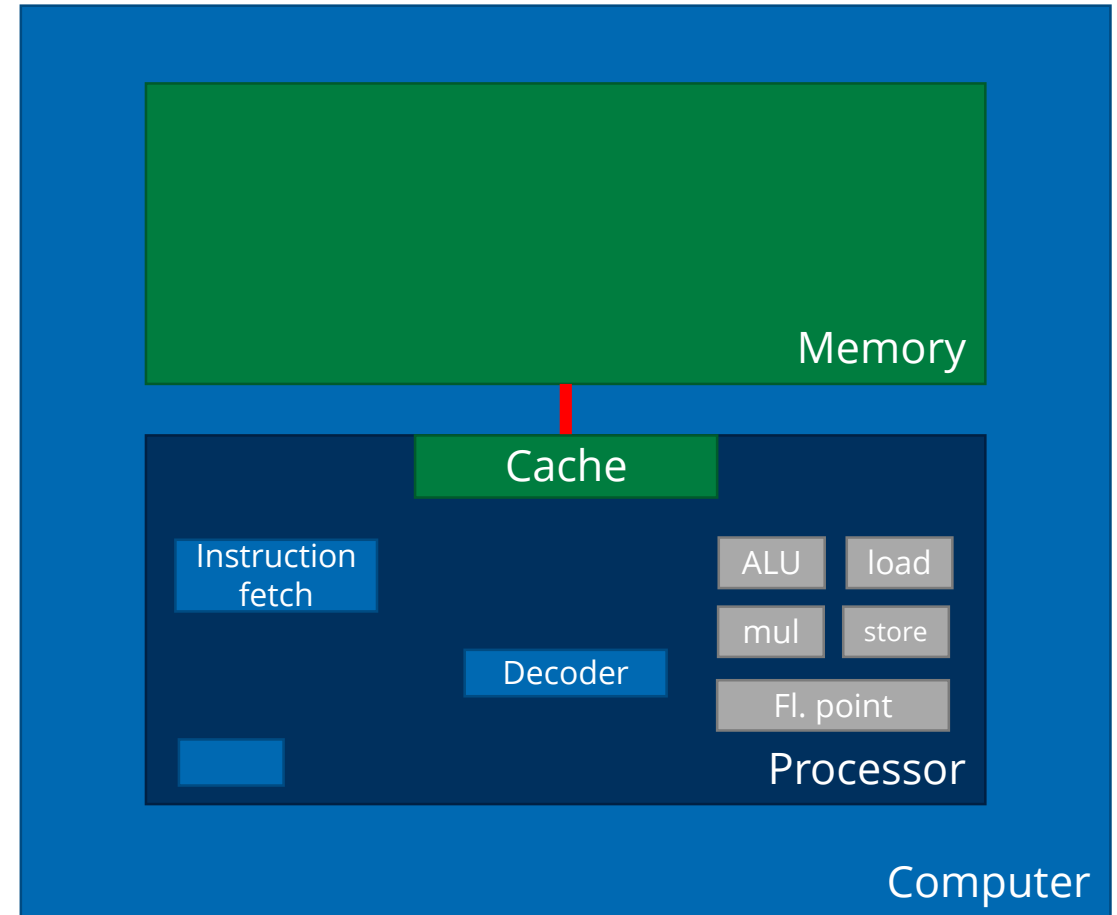
Problem

- Memory access via bus is slow
- How slow?
- > 100 cycles for one access
- Only limited number of registers
 - have to use memory



Principle

- Buffer recently used parts of memory
- Buffer memory that is close to recently used parts of memory
- Use fast buffer, which resides in processor
- Accesses to data which is (close to) recently used are fast
- Accesses to non-recently-used data are slow



Details

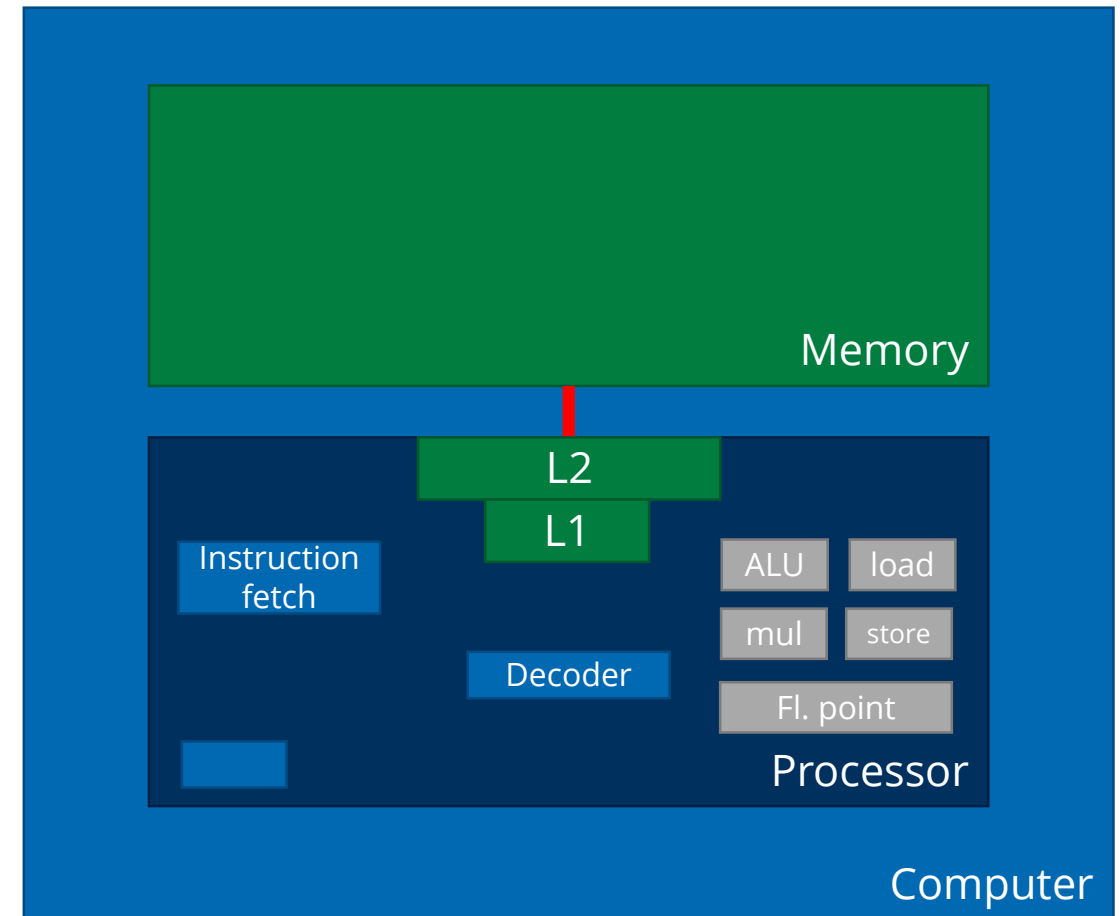
- Cache holds multiple lines of data (cache line)
- Cache line
 - typically 64 byte of consecutive memory per line (64 byte == 8 x 64 bit values/16 x 32 bit values/...)
 - In addition to data, the line holds information about the memory address which it holds a copy of
 - Can be clean (1:1 copy of memory)
 - Can be dirty (changes to cache line but not to memory)
- Caches have limited space
 - For each new cache line added to cache, an old cache line must be evicted
 - If dirty, then write back to memory

addr	int64	int64	int64	int64
addr	int64	int64	int64	int64
addr	int64	int64	int64	int64
addr	int64	int64	int64	int64

addr	int64	int64	int64	int64
------	-------	-------	-------	-------

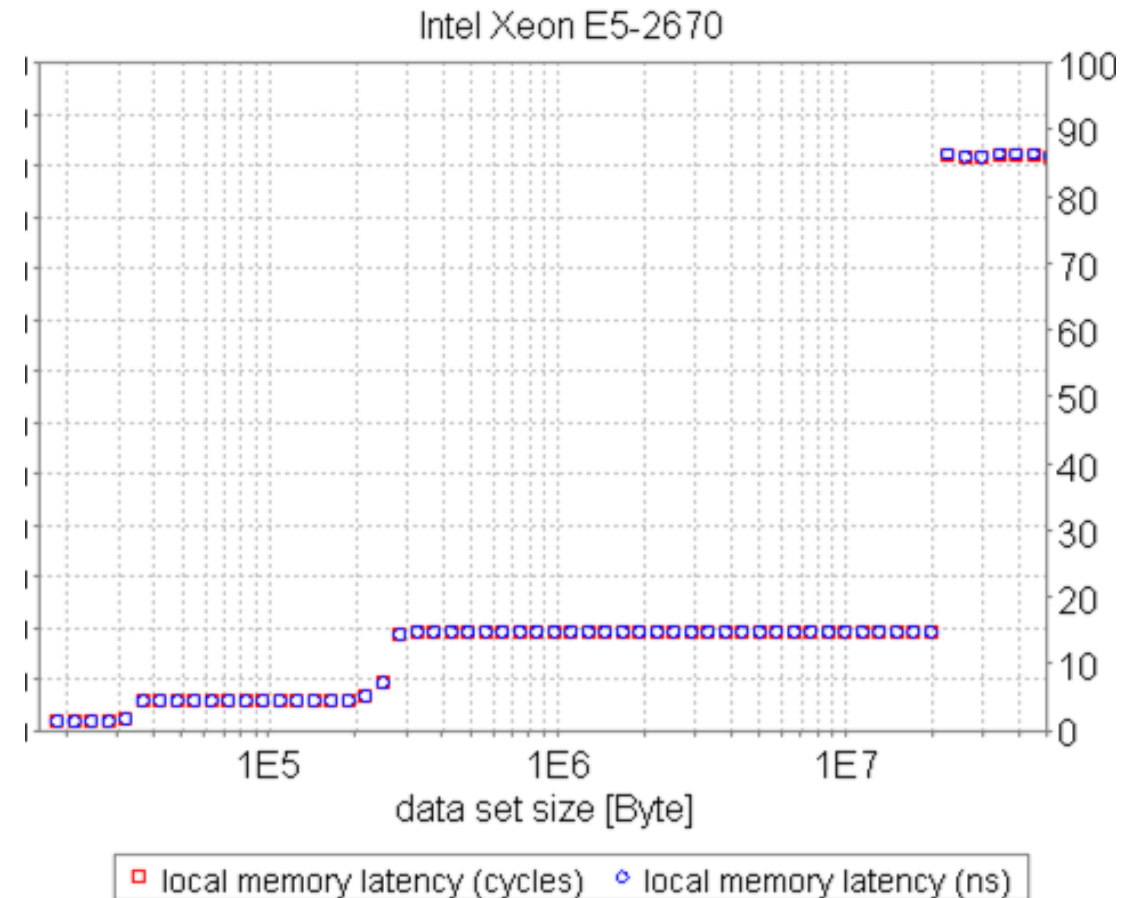
Details

- Multiple cache levels (L1, L2, L3)
 - Increasing size
 - Increasing latency (how long until data is read)
- Example 2 Caches, read 64 bit from address
 - 1. Check L1 cache
 - If present select line and return the corresponding 64 bit
 - 2. Check L2 cache
 - If present select line and return the corresponding 64 bit
 - 3. Read from memory



Speedup

- Depends on the latency and bandwidth of the cache
- Typical latencies:
 - L1 cache: 2-4 cycles
 - L2 cache: ~10 cycles
 - L3 cache: ~40 cycles
 - RAM: ~250 cycles (very rough estimate)



Instruction Pipelining

Problem

- A single instruction has multiple steps, a typical example* would be:
Instruction Fetch, Instruction Decode, Execute, Memory Access, Writeback to register
- Each of these parts has specialized hardware
- All of this hardware is used only 1/5th of the time and 4/5th it just idles

Cycle 0: • Read PC • Access memory • Read instruction	Cycle 1: • Read instruction • Select execution unit • Pass instruction	Cycle 2: • Process instruction	Cycle 3: • Access memory	Cycle 4: • Write result to registers
---	--	--	--	--

*(Actual pipeline is highly processor-dependent)

Idea

- Use these units in parallel
- After one unit finishes, it can start working on the next instruction
- Cycle 0:
 - Fetch instruction 0
- Cycle 1:
 - Decode instruction 0
 - Fetch instruction 1
- Cycle 2:
 - Execute instruction 0
 - Decode instruction 1
 - Fetch instruction 2



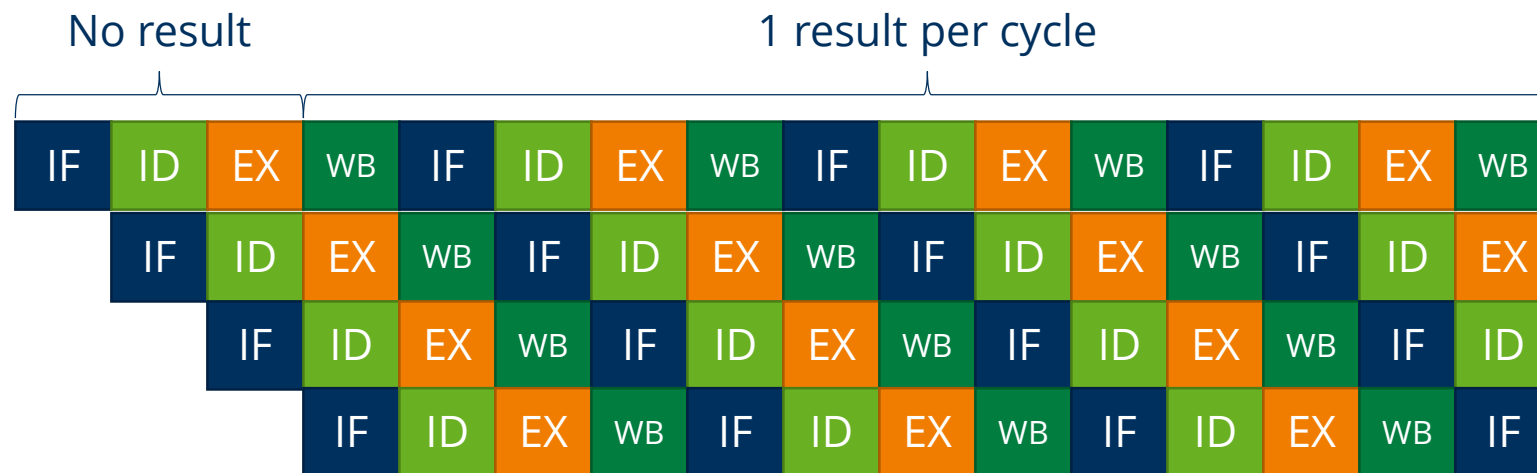
New Problem: Hazards

Pipeline stalls (hazards) if:

- Instructions depend on each other ($a=b+c$; $d=2*a$;) —
- There's a branch/jump instruction (which way to go?) —
- Multiple stages use common resources (IF, MEM both use the memory controller) —
- ...

Speedup

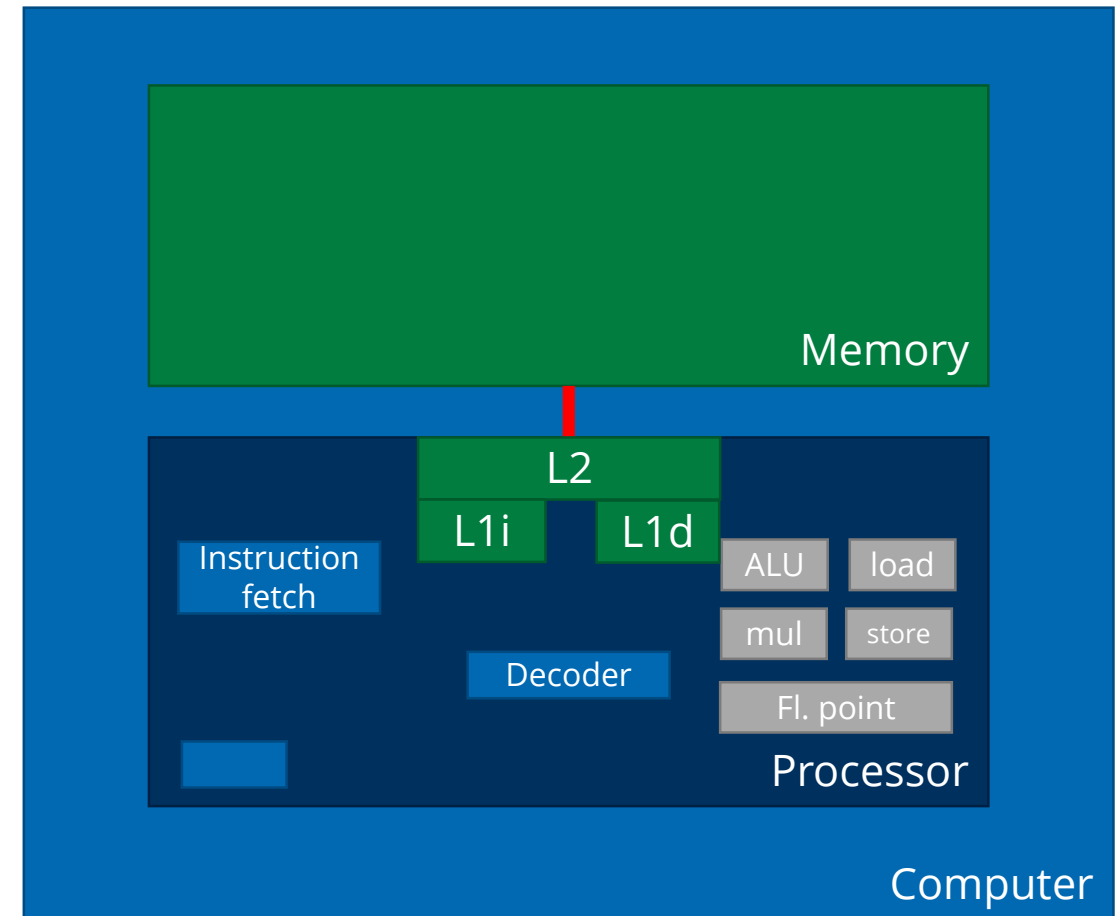
- Depends on pipeline length (k)
- First result after k cycles
- Afterwards, one result every cycle
- Probably lower due to hazards
- Speedup: $S_k = \frac{T_1}{T_k} = \frac{(k*n)}{n+(k-1)}$, for $\lim_{n \rightarrow \infty} S_k = k$



Harvard Architecture

Problem and solution

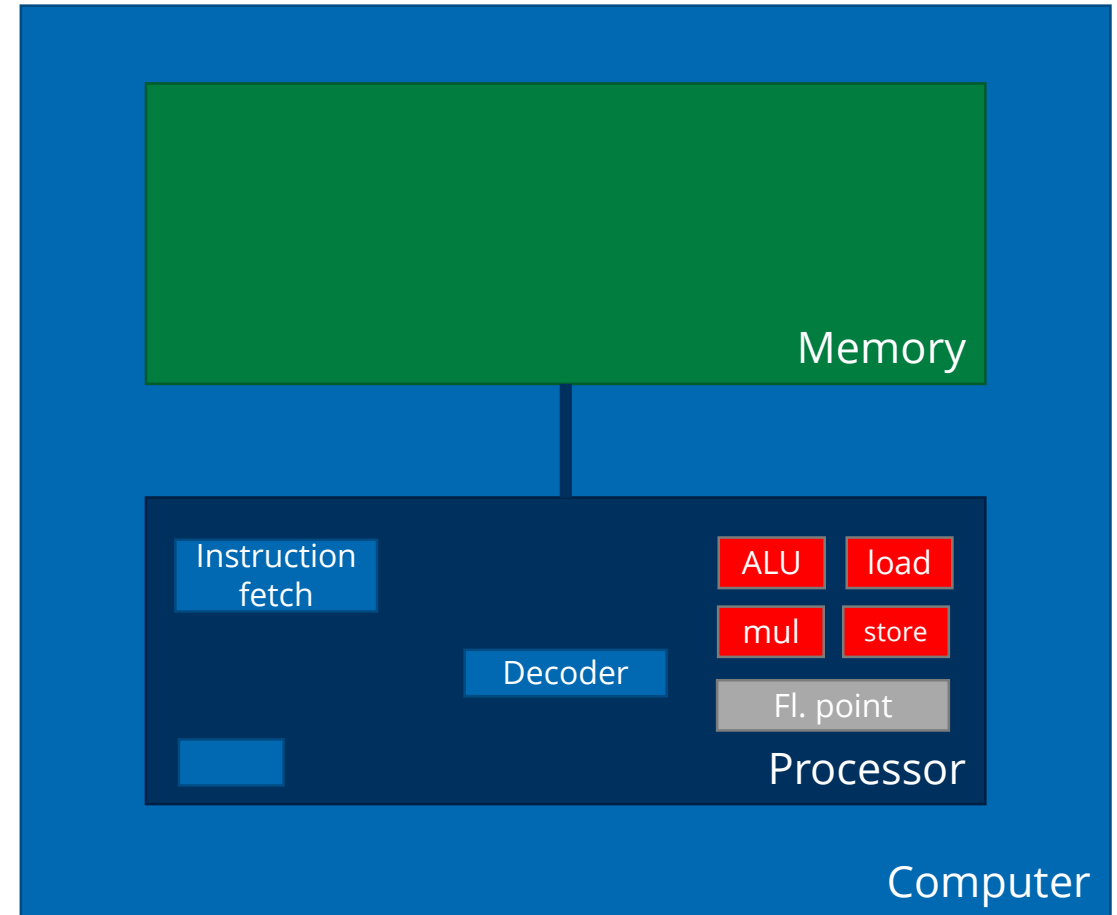
- Instruction Fetch and Memory Access access memory for instructions and data, resp.
- Needs high bandwidth, low latencies
- Shared resources: memory controller, busses, memory
- Solution: Harvard Architecture
separate memories for instructions and data
→ can serve instructions and data in parallel
- Today used in L1 caches, split into L1i and L1d



Superscalarity

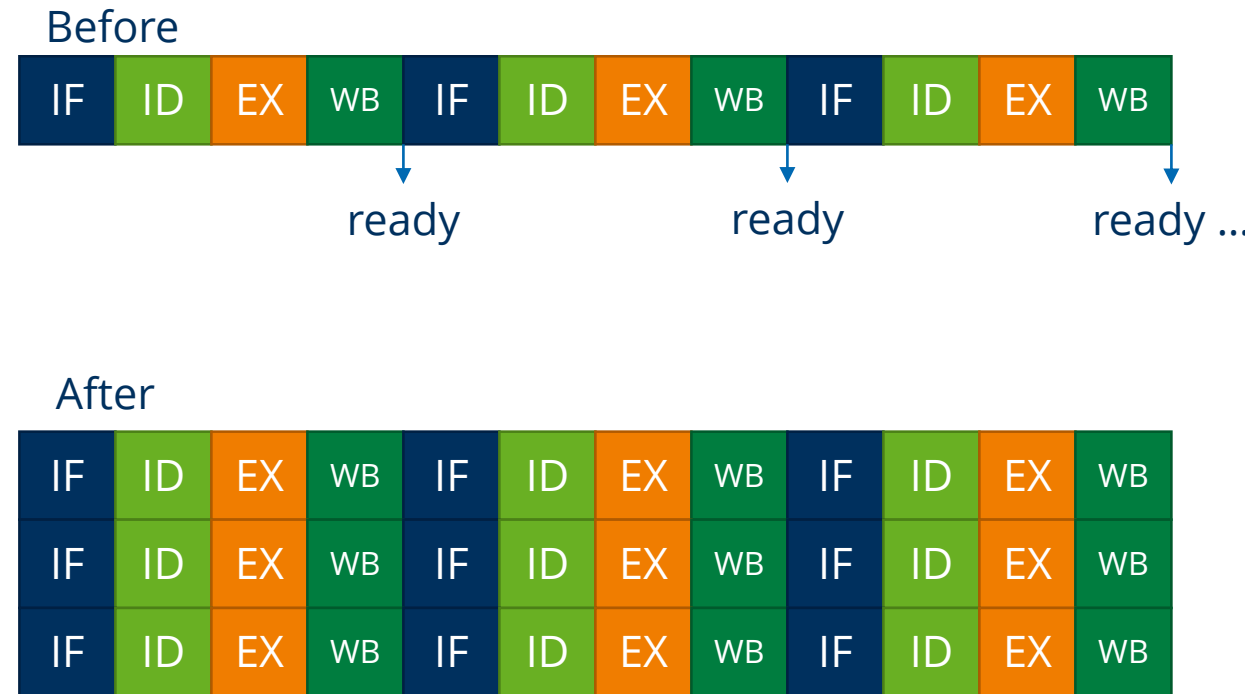
Problem

- Most of the time Execution Units not used (during floating point addition, ALUs not used)
- Inefficient use of transistors



Solution and Speedup

- Instruction Fetch/Decode can fetch/decode multiple instructions
- Pass them to multiple Execution Units, if:
 - there are enough EUs available
 - instructions do not depend on each other
- Speedup depends on:
 - Number of instructions that can be fetched/decoded
 - The number and types of EUs
 - The instruction mix
- 3x superscalar $\rightarrow$ max. speedup = 3
- Can be combined with pipelining



Out-of-Order Execution, Branch Prediction

Out-of-Order Execution

- Default: still have to wait if a single instruction stalls (e.g., due to memory access)
- What if there were instructions that could be executed?
- De-couple IF/ID with EX/WB
- Collect instructions
 - Mark them ready if their data is available
 - If ready, pass them to execution units
- Needs significant amount of processor space and management overhead
- According to ISA: Everything is still in-order!

Branch Prediction

- Every 8th instruction is a branch
- Pipelining needs to stop if if-clause depends on data that is to be computed
- Branch prediction
 - Take notes on whether a branch was taken previously
 - Take branch based on previous behavior
 - Good for for-loops
 - Simple: Do what was done last time
 - Complicated: Take note for each branch instruction, also store possible target address

The Operating System

Role: Manage Access to Resources

- Provide common interface for hardware
 - Example file system: No matter your file system (FAT, EXT3, ZFS, NFS), you'll see folders and files. Users usually do not write a byte stream to a location on a hard drive
- Provide access limitation
 - Example file system: Ensure file access rights
 - Example running programs: Users only have control over their programs
 - Only some users/programs may use specific hardware
- Manage access to hardware
 - Sequentialize parallel access of programs to a single device, e.g., network card
 - Schedule processes fair on available CPUs
- OS runs hardware in privileged mode. Is typically executed on same hardware (processor cores) as user software.

Multi Program Environments

- Problem 1: If there are two programs, they cannot use the same physical memory if they do not want to change the others data → Cannot use same addresses
- Problem 2: A user program should not be able to access memory from another program or the operating system → Cannot use same addresses
- Problem 3: If each program is allowed to allocate the whole memory, the installed memory is not sufficient if the number of programs is to high.
- Problem 4: Splitting the available memory among programs is mostly never fair

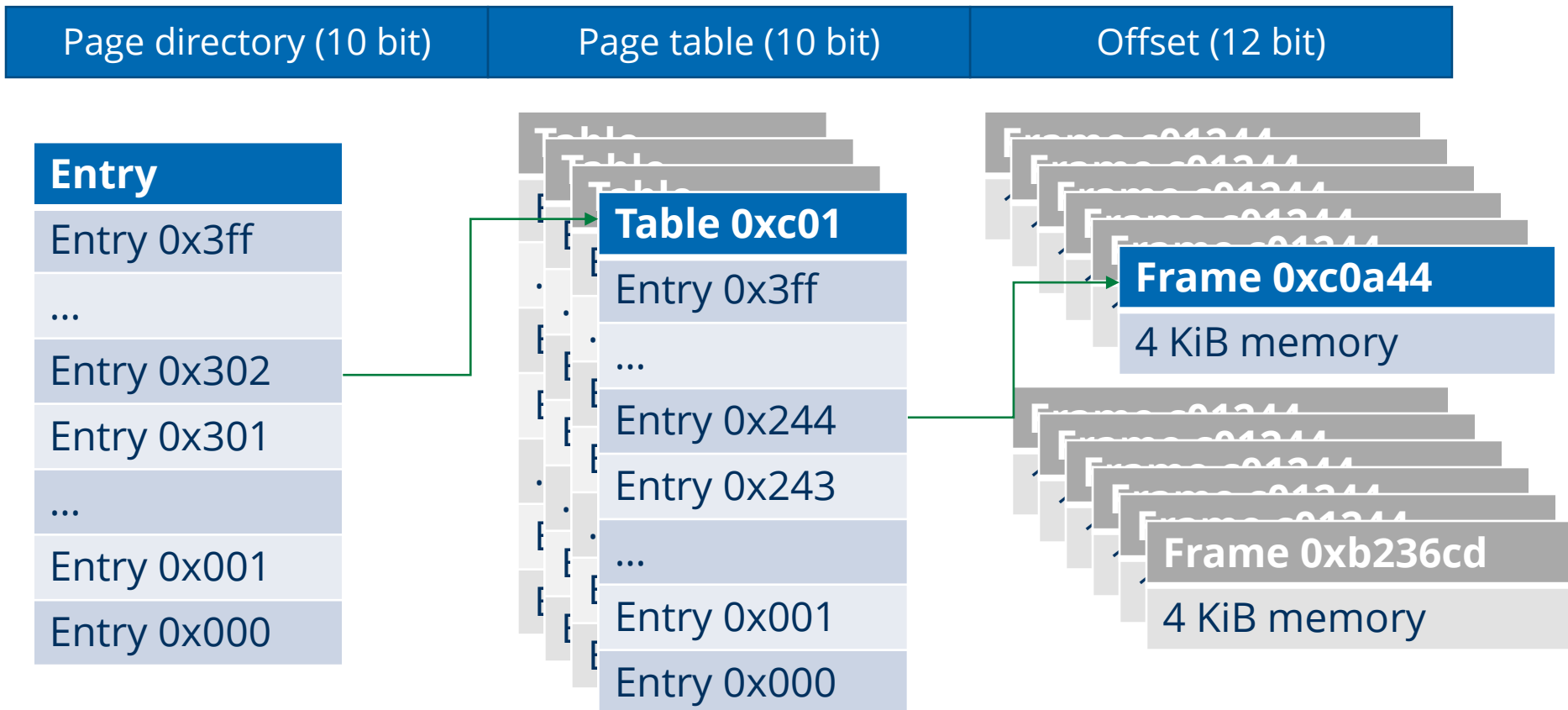
➤ Virtual Memory

Virtual Memory

- The operating system hands out virtual addresses when memory is allocated
- Virtual addresses do not directly relate to physical addresses
- Memory is given to the program with a given page-granularity (e.g., 4 KiB)
- There are tables where each page is located in physical memory
- When a process accesses memory, the virtual address is translated to a physical address
 - Page fault
 - Handled by hardware and operating system
- This process can have multiple stages!
- The operating system will randomize parts of the virtual addresses for more security
- When main memory does not suffice, pages can be written to hard drive

Virtual Memory

Address: 0xc0a4412f – split into 0x302 (page directory entry), 0x244 (page table entry), 0x12f (offset)



Switching Programs on a Single Processor

- Each program has own Page Directory/Page Tables → separate physical memory
- Each program has own instruction pointer
- Each program has own registers
- ...
- Operating system stores internal state (registers) when switching to another program
- Operating system restores internal state when reactivating program
- Preemption



Starting a New Program

- Needs own address space
- Operating system creates address space, loads executable from HDD
- Operating system sets pointer to first instruction
- New program runs, old program is ended/interrupted

Starting a New Process Within a Program

- Also called a *fork*
- Copies address space
- Inherits all the file handles (open files, communication channels, ...)
- Can start a new program
- Old process runs concurrently
- After fork:
 - Changes to own memory not visible to other process
 - Newly open file handles not visible to other process
 - Independent programs → Communication via file handles (e.g., files, network, ...)
- Parent process can wait for child process to end
- Each program (job) consists of one or multiple processes



Simple Example

```
int a = 2 ;
int b = 3 ;
void main() {
pid_t pid = fork(); // simple, so no error handling
    if ( pid == 0 ) { // child
        int c = a + b;
        printf("a+b=%d\n", c );
    }
    else { // parent
        a=4; // change value of variable
        int c = a + b ;
        printf("a+b=%d\n", c );
    }
}
```

Communication between Processes

- Example 1 : Web Browser
 - open a socket for http communication
 - send web request
 - receive HTML page
- Example 2 : Web Server
 - open a socket for http communication
 - receive web request
 - send HTML page
- Example 3: `ls -la | grep "*.mp3"`
 - Creates two processes: `ls` and `grep`, connected via a pipe
 - `ls` writes file list to its `stdout` filehandle
 - `grep` reads from its `stdin` filehandle

Starting a New Thread Within a Program

- Sometime also called a *fork*
- New thread gets own instruction pointer and stack memory
- Heap memory and file handles used concurrently
- Each process has one or multiple threads
- Communication via shared (heap) memory
- Problem concurrent access: Both threads access a single variable

```
shared_variable = shared_variable + 1;
```

Small Reminder

- **We are still sequential (from an ISA point of view)**
- We are only **in software** parallel with multiple programs/processes/threads(from software point of view), but still on one core
- **Parallelism only in software**, some hardware extensions necessary
- Operating system switches processes/threads on a single CPU
- We still have not talked about “real parallelism”
- That’ll be in the lecture